
Structure of Android Tasks and Services

- 9.1 Learning Outcomes
- 9.2 AsyncTask
- 9.3 Location-Based Services
- 9.4 Self-Assessment Questions
- 9.5 Self-Assessment Activities
- 9.6 Multiple-Choice Questions
- 9.7 Key answers to multiple-choice questions
- 9.8 Summary
- 9.9 Keywords
- 9.10 Recommended resources for further reading

9.1 Learning Outcomes:

After the successful completion of this chapter, the student will be able to:

- Describe AsyncTask Class used in Android for background processing
- Demonstrate the use of AsyncTask in Android with ProgressBar Updates
- Describe the use of AsyncTask to send HTTP requests in the background
- Describe location-based services in Android

9.2 AsyncTask

9.2.1 AsyncTask Overview and methods

In Android, application code statements are executed in the main thread. Sometimes a task may require more time to process which may block the main thread until the operation is performed and completed. In such cases, the tasks that require more time and do not need user interaction can be performed in the background by using another thread. Asynchronous task (AsyncTask) in android allows, execution of some of the instructions in the background using a background thread and later again synchronizing with the main thread. The results of the background thread are later passed to the main thread, this makes the main user interface thread light and the application more responsive.

AsyncTask is a helper class in android used to perform background processing.

An AsyncTask class in android consists of 3 generic types

- **TypeOfVarArgParams** - It contains the information about the type of parameters sent to the task during execution
- **ProgressValue** - When the background processing is going on, the updates of it are sent to the user interface. It contains information about the progress
- **ResultValue** - The result information is stored in it.

The general syntax is as follows. In case only some generic types are used by the asynchronous Task class then it is marked as unused by using the type void.

```
private class MyBGTask extends AsyncTask<Void, Integer, String>
{
    ...
}
```

AsyncTask contains 4 methods which are executed as shown in figure 9.1

1. **onPreExecute()**
2. **doInBackground()**
3. **onProgressUpdate()**
4. **onPostExecute()**

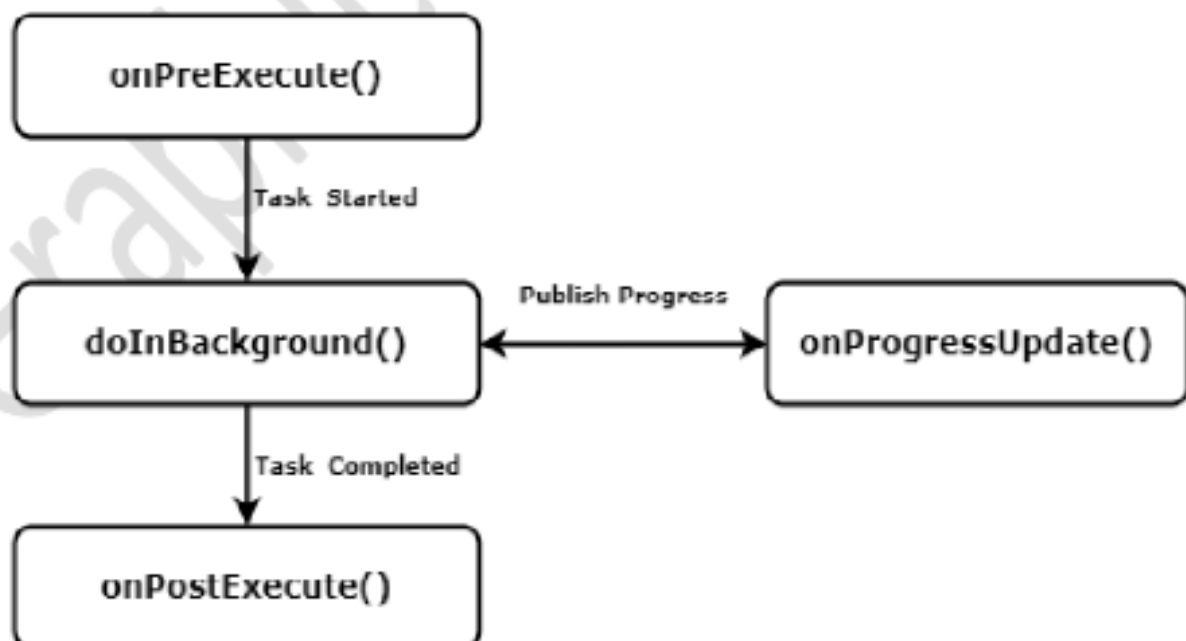


Figure 9.1: AsyncTask methods execution process

1. **onPreExecute()** - This method is invoked before starting the background operation. In this method usually, some instance of ProgressBar/ProgressDialog or animation is created and shown in the user interface to notify the user that some background operation is about to start.
2. **doInBackground()** - In this method background operations instructions are executed in the background thread. This method is executed immediately after onPreExecute()
3. **onProgressUpdate()** - This method is used to update the user interface i.e., the instance of ProgressBar/ProgressDialog or animation set up in the **onPreExecute()** method while the background operations are still going on.
4. **onPostExecute()** - This method is invoked after the background operations are completed. The user interface is updated to show the result.

Sample of Custom class extended from AsyncTask class with methods

```
private class MyBGTask extends AsyncTask<Void, Integer, String>
{
    @Override
    protected void onPreExecute() {
        // Task to be performed before starting background task
    }
    @Override
    protected String doInBackground(Void... voids) {
        // execute a background task pass value to onProgressUpdate()
        publishProgress(integer-value);
        return "xxx";
    }
    @Override
    protected void onProgressUpdate(Integer... values) {
        // show the progress of the background task
    }
    @Override
    protected void onPostExecute(String result) {
        // executed on completion of background task
    }
}
```

Figure 9.2: Sample of Custom class extended from AsyncTask with methods

9.2.2 Implementation of AsyncTask with example

Example of [AsyncTask](#) with thread executing in the background and showing the progress of the process in [ProgressBar](#)

Create a project in Android and add a Button and [ProgressBar](#) to the activity_main.xml file as shown here.

```
<ProgressBar
    android:id="@+id/prgbar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:minHeight="20dip"
    android:maxHeight="20dip"
    style="@android:style/Widget.ProgressBar.Horizontal" >
</ProgressBar>
```

Figure 9.3: Snippet of activity_main.xml file with ProgressBar

Make changes in the MainActivity.java file to execute the [AsyncTask](#) in the background and show the progress of the [background](#) process in [ProgressBar](#)

```
public class MainActivity extends AppCompatActivity {
    ..
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        // get the references to the button and ProgressBar
        btn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                AsyncTaskExample asyncobj = new AsyncTaskExample();
                asyncobj.execute();
            }
        });
    }
    private final class AsyncTaskExample extends
        AsyncTask<Void, Integer, String> {
        @Override
```

```

protected void onPreExecute() {
    super.onPreExecute();
    prgbar.setVisibility(View.VISIBLE);
    prgbar.setMax(100);
    prgbar.setProgress(0);
}
@Override
protected String doInBackground(Void... voids) {
    for (int i = 0; i < 10; i++) {
        try {
            Thread.sleep(1000);
            publishProgress(i*10);
        } catch (InterruptedException e) {
            e.printStackTrace();    // Interrupted
        }
    }
    return "completed";
}
@Override
protected void onProgressUpdate(Integer... values) {
    prgbar.setProgress(values[0]);
}
@Override
protected void onPostExecute(String result) {
    if (result != null) {
        Toast.makeText(MainActivity.this, "Background Task : "+
            result, Toast.LENGTH_LONG).show();
    } else
        Toast.makeText(MainActivity.this, "Background Task
            incomplete", Toast.LENGTH_LONG).show();
    prgbar.setVisibility(View.GONE);
}
}
}

```

Figure 9.4: Snippet of MainActivity.java with AsyncTask implementation

On Executing the status of the ProgressBar is shown in the display

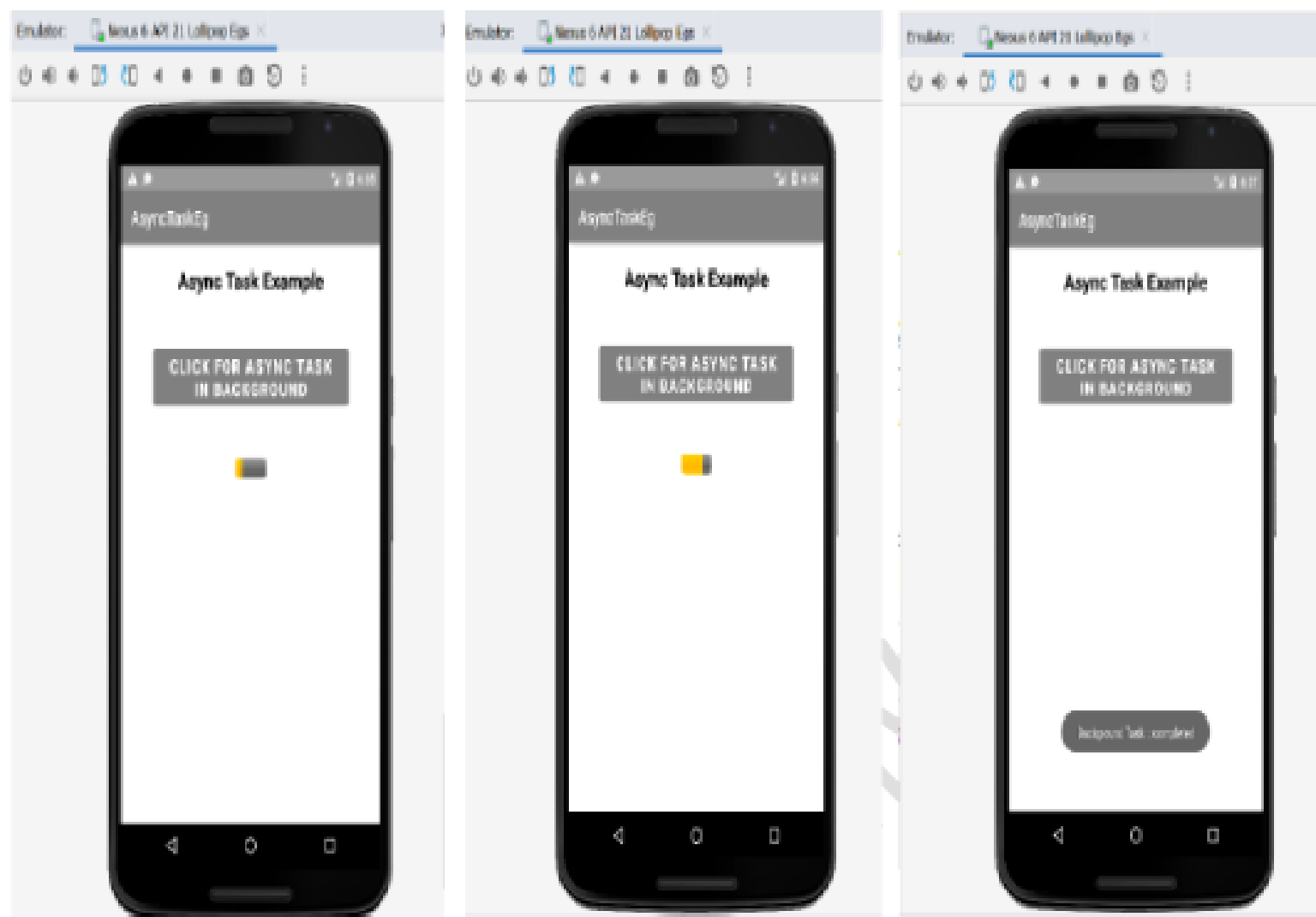


Figure 9.5: AsyncTask with ProgressBar showing progress and completion of Task

9.2.3 AsyncTask using HTTP

To send an HTTP request in Android the following classes are used in Android

- [HttpClient](#)
- [AndroidHttpClient](#)
- [HttpURLConnection](#)

To send an HTTP Request we need to

- Declare a URL Connection and create a URL object
- Open the connection by creating an object of [HttpURLConnection](#) class
- Once the connection is successful, download and decode the data based on its type
- Wrap in [AsyncTask](#) Class object and execute it in the background thread

Sample Code to download an image from a specific site and display it

1. Declare a URL Connection and create a URL object

```
URL urlobj = new URL("https://xxx.com/imagefile.jpg");
```

2. Open the HTTP connection by creating an object of HttpURLConnection class

```
HttpURLConnection con = (HttpURLConnection)  
    urlobj.openConnection();  
  
con.connect();
```

```
// Open InputStream to connection
```

```
InputStream ins = con.getInputStream();
```

3. Download the image and decode using BitmapFactory

```
Bitmap bmpobj = BitmapFactory.decodeStream(ins);  
ins.close();
```

4. Insert the downloaded image in the ImageView control for display

```
ImageView imgobj = (ImageView) findViewById(R.id.imageview1);  
Imgobj.setImageBitmap(bmpobj);
```

Figure 9.6: Snippet of AsyncTask using HTTP Connection to download Image

9.3 Location-Based Services

Location-based services(LBS) in android are very popular nowadays. LBS applications help to track the device location and locate nearby amenities like hospitals, hotels, restaurants, stores, pharmacies, tourist places, etc. They help in finding routes and offer suggestions regarding route planning. Cab drivers nowadays use google maps to find routes. They also provide features of geofencing

By accessing Google Play Services and Location Framework in android, location-based applications can be developed easily.

9.3.1 Displaying Google Maps

For integrating google maps in an android application, Google Play Services are used. The Maps API key has to be obtained by the process of registration and then used in the android application for displaying the map.

Google Maps display the current location, search location, navigation direction, etc.

There are 4 types of google maps

1. Normal - This map displays rivers, roads, mountains, buildings, etc.
2. Hybrid - This map displays data received through satellite photographs along with roads and features that are labeled.
3. Satellite - This map displays only data received through satellite photographs and does not display road labels.
4. Terrain - This map displays photographic information which includes labels, lines, colors, perspective shading, etc.

Google Map API provides methods that are used for getting and customizing google maps

9.3.2 Getting Location data

Location data in android is fetched using GPS receivers, Cellular networks (Cell tower triangulation), and Wi-Fi.

GPS receivers use satellites to easily find the location. But in case of the unclear sky and in places like tunnels through mountains where satellites can't penetrate it becomes a problem to find the location.

Cellular networks work best when the cell towers are located very close to each other. Knowing the cell tower's identity it is easy to get the location and its information.

Wi-Fi can also be used to find the location

For getting the location data the following classes and interfaces are used in an android application

LocationManager class – provides Location Services for the system

LocationListener interface – notifications/updates from LocationManager class are received using this interface

LocationProvider class - provides device location

Location class – It provided information about the location like longitude and latitude etc.

For e.g. getLongitude(), getLatitude(), and getAltitude() methods are available in this class.

Geocoders provide 2 types of services for getting location data

Forward Geocoding - translating address information to longitude and latitude

Reverse Geocoding – translating from longitude and latitude to address form

9.4 Self-Assessment Questions

- Q1. Give an overview of AsyncTask Class and its methods.[8 marks, L2]
- Q2. Illustrate with suitable code the use of AsyncTask to run threads in the background and show the progress of the process in ProgressBar.[8 marks, L2]
- Q3. Discuss AsyncTask used with HTTP request in brief. [6 marks, L2]
- Q4. Explain location-based services in android. [8 marks, L2]
- Q5. Write a note on [5 marks each, L2]
- Google maps in android studio
 - Getting location data

9.5 Self-Assessment Activities

Note: Use an appropriate emulator for the execution of an android application.

- A1. Create an Android application, and add appropriate controls. Demonstrate the use of AsyncTask with thread executing in the background and showing the progress of the process in ProgressBar. The output should be as shown in figure 9.5.

9.6 Multiple-Choice Questions

1. The class used in Android to perform background processing is _____. [1 mark, L1]
 - a) SyncTask
 - b) AsyncTask
 - c) BackgroundTask
 - d) None of the above
2. The method which gets executed before the background process starts is _____. [1 mark, L1]
 - a) onPreExecute()
 - b) PreExecute()
 - c) beforestart()
 - d) None of the above
3. The method which does not belong to AsyncTask Class is _____. [1 mark, L1]
 - a) onPreExecute()
 - b) doInBackground()
 - c) onEndExecute()
 - d) onPostExecute()

4. The AsyncTask method in which background operations instructions are executed in the background thread is _____. [1 mark, L1]
- a) onPreExecute()
 - b) doInBackground()
 - c) onProgressUpdate()
 - d) onPostExecute()
5. To send an HTTP request the class used in Android is _____. [1 mark, L1]
- a) HttpURLConnection()
 - b) HTTPClient
 - c) HTTPServer
 - d) HttpURLConnection
6. To use google maps in the android studio we need to use _____. [1 mark, L1]
- a) Google Maps API
 - b) Google APIs
 - c) Google Play Services
 - d) None of the above
7. The type of map that displays satellite photograph data with typical road maps and feature labels in android is _____. [1 mark, L1]
- a) Hybrid
 - b) Normal
 - c) Satellite
 - d) Terrain
8. The type of map that displays a typical road map, natural features like rivers, and some features built by humans in the android is _____. [1 mark, L1]
- a) Hybrid
 - b) Normal
 - c) Satellite
 - d) Terrain
9. The methods getLongitude() and getLatitude() belong to the class _____. [1 mark, L1]
- a) LocationManager

- b) LocationProvider
- c) Location
- d) None of the above

10. Translating address information to longitude and latitude in android is known as _____. [1 mark, L1]

- a) Forward Geocoding
- b) Reverse Geocoding
- c) Translation
- d) GPS receiver

9.7 Key Answers to multiple-choice questions

1. The class used in Android to perform background processing is AsyncTask. [b]
2. The method which gets executed before the background process starts is onPreExecute(). [a]
3. The method which does not belong to AsyncTask Class is onEndExecute().[c]
4. The AsyncTask method in which background operations instructions are executed in the background thread is doInBackground(). [b]
5. To send an HTTP request the class used in Android is HttpURLConnection.[d]
6. To use google maps in the android studio we need to use Google Play Services.[c]
7. The type of map that displays satellite photograph data with typical road maps and feature labels in android is Hybrid.[a]
8. The type of map that displays a typical road map, natural features like rivers, and some features built by humans in the android is Normal.[b]
9. The methods getLongitude() and getLatitude() belong to the class Location.[c]
10. Translating address information to longitude and latitude in android is known as Forward Geocoding.[a]

9.8 Summary

Asynchronous task (AsyncTask) in android allows, execution of some of the instructions in the background using a background thread and later again synchronizing with the main thread. The results of the background thread are later passed to the main thread, this makes the main user interface thread light and the application more responsive.

AsyncTask is a helper class in android used to perform background processing.

An AsyncTask class in android consists of 3 generic types -TypeOfVarArgParams, ProgressValue, and ResultValue.

AsyncTask contains 4 methods - **onPreExecute()**, **doInBackground()**, **onProgressUpdate()** and **onPostExecute()**.

To send an HTTP request in Android the following classes are used in Android – HttpClient, AndroidHttpClient, and HttpURLConnection.

Location-based services(LBS) in android are very popular nowadays. LBS applications help to track the device location and locate nearby amenities like hospitals, hotels, restaurants, stores, pharmacies, tourist places, etc. They help in finding routes and offer suggestions regarding route planning. They also provide features of geofencing.

By accessing Google Play Services and Location Framework in android, location-based applications can be developed easily.

For integrating google maps in an android application, Google Play Services are used.

The Maps API key has to be obtained by the process of registration and then used in the android application for displaying the map.

Google Maps display the current location, search location, navigation direction, etc.

There are 4 types of google maps - Normal, Hybrid, Satellite, and Terrain.

Google Map API provides methods that are used for getting and customizing google maps

Location data in android is fetched using GPS receivers, Cellular networks (Cell tower triangulation), and Wi-Fi.

9.9 Keywords

- AsyncTask
- ProgressBar
- HttpURLConnection
- Google maps
- Terrain
- Location-Based Services
- GPS receivers
- Cellular networks (Cell tower triangulation)
- Wi-Fi.
- Forward Geocoding
- Reverse Geocoding

9.10 Recommended Resources for Further Reading

Text Book(s)

1. Pradeep Kothari, "Android Application Development" Black-Book, Dreamtech Press, 2015
2. Wei-Meng Lee, "Beginning Android Application Development", Wiley 2011.
3. Dawn Griffiths and David Griffiths, "Head First Android Development", O'Reilly, 2017.

References

- <https://www.javatpoint.com/android-tutorial>
- <https://www.tutorialspoint.com/android/index.htm>
- <https://developer.android.com/guide/components/fundamentals>

Contents

Topics	Page No.
Unit-4: Android Tasks, Services, and Testing	1
Chapter-10: Testing Android Applications	
10.0 Structure of Testing Android Applications	1
10.1 Learning Outcomes	1
10.2 Testing Android Apps.....	1
10.2.1 Levels of Testing.....	1
10.2.2 Types of Testing.....	6
10.3 Android Testing Frameworks.....	6
10.4 Self-Assessment Questions.....	7
10.5 Self-Assessment Activities.....	7
10.6 Multiple-Choice Questions.....	7
10.7 Key Answers to Multiple-Choice Questions.....	9
10.8 Summary	9
10.9 Keywords.....	10
10.10 Recommended Resources for Further Reading.....	10

UNIT 4 - Android Tasks, Services, and Testing

CHAPTER 10 –Testing Android Applications

Structure of Android Tasks and Services

- 10.1 Learning Outcomes
 - 10.2 Testing Android Apps
 - 10.3 Android Testing Frameworks
 - 10.4 Self-Assessment Questions
 - 10.5 Self-Assessment Activities
 - 10.6 Multiple-Choice Questions
 - 10.7 Key answers to multiple-choice questions
 - 10.8 Summary
 - 10.9 Keywords
 - 10.10 Recommended resources for further reading
-

10.1 Learning Outcomes

After the successful completion of this chapter, the student will be able to:

- Describe the levels/types of testing used in android application development
 - Differentiate between unit, integration, system, and acceptance testing
 - Describe various tools used to test an android application
-

10.2 Testing Android Apps

Any software development is incomplete without testing. In a software application development process, testing is one of its phases. Any software application developed is tested before it is handed over to the end user. Testing is done to find out if any error(s) exists. The errors identified are fixed and checked to deliver an application of quality to the users. To assure and maintain the quality of the software, the application is tested at various levels as shown in figure 10.1.

10.2.1 Levels of Testing

The different levels of testing are

- Unit Testing
- Integration Testing

- System Testing
- Acceptance Testing

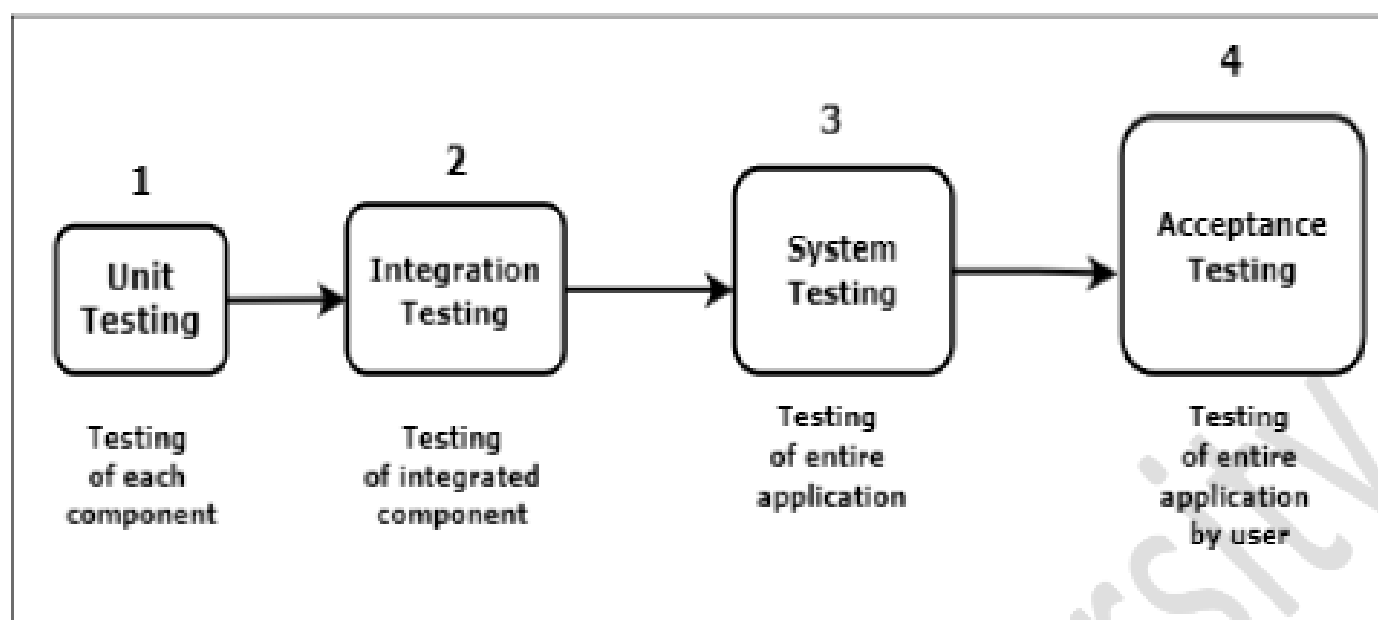


Figure 10.1: Levels of software testing

Unit Testing - Testing of small unit/program code is called as unit testing. In the case of Android applications For e.g. Consider a login Activity that allows the user to choose either of the radio buttons

- Create a New account
- Sign In

Test Case 1: On Click of the 1st radio button Create a New account, and a registration form for new customers should appear as shown in figure 10.3. The `onClick()` event is tested to check whether the event handler code is calling the next appropriate activity as required. In this case, whether the Registration form for a New customer gets displayed.

The following code snippet shows the eventListener added to the radio button and on click, an Intent object is created and the next activity is called.

```

// For new customer
RBnewlogin.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {    // Unit testing
        // creating Intent object
        Intent intent=new Intent(LoginActivity.this,LoginNewActivity.class);
        startActivity(intent);    // starting activity for new customer
    }
});
  
```

Figure 10.2:Snippet of `onClick()` for unit testing

[Click of radio button for the new customer]

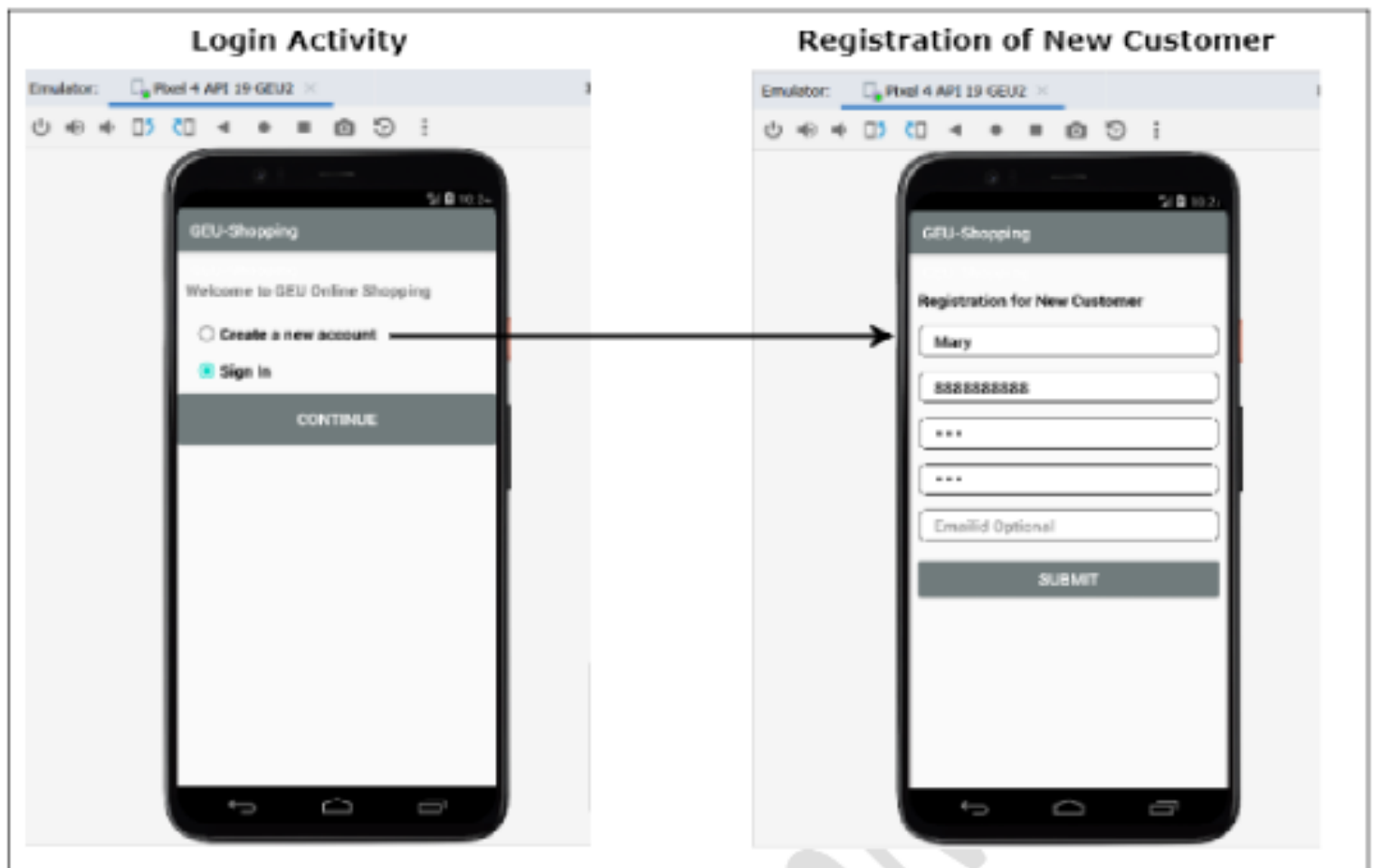


Figure 10.3: Display of the Registration form for new customers on Click of the radio button

Test Case 2: On Click of the 2nd radio button Sign In, a registration form for the registered customer should appear as shown in figure 10.5. The `onClick()` event is tested to check whether the eventhandler code is calling the next appropriate activity as required. In this case, whether the Registration form for a Registered customer gets displayed.

The following code snippet shows the `EventListener` added to the radio button and `onClick`, an `Intent` object is created and the next activity is called.

```
// For registered customer
RBregdusr.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {    // Unit testing
        // creating Intent object
        Intent intent=new Intent(LoginActivity.this,LoginRegdActivity.class);
        startActivity(intent);    // starting activity for registered customer
    }
});
```

Figure 10.4:Snippet of `onClick()` for unit testing

[Click of radio button for the registered customer]

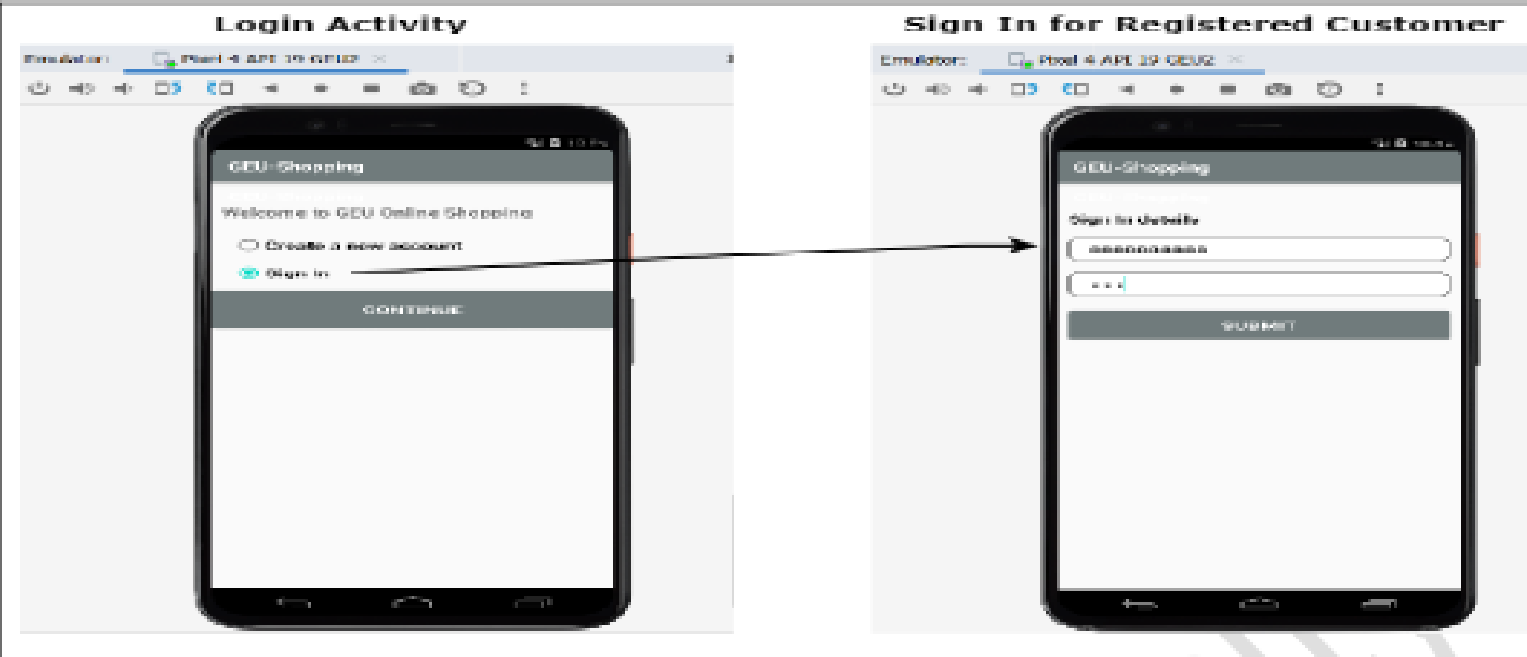


Figure 10.5:Display of the Registration form for registered customers on Click of the radio button

Integration Testing: After testing the small modules, these modules are integrated and then tested to check whether the code gets executed properly. For e.g. on the Click of 1st radio button Create a New account, a registration form for the new customer should appear, and on Click of 2nd radio button Sign In, a registration form for the registered customer should appear as shown in figure 10.6.

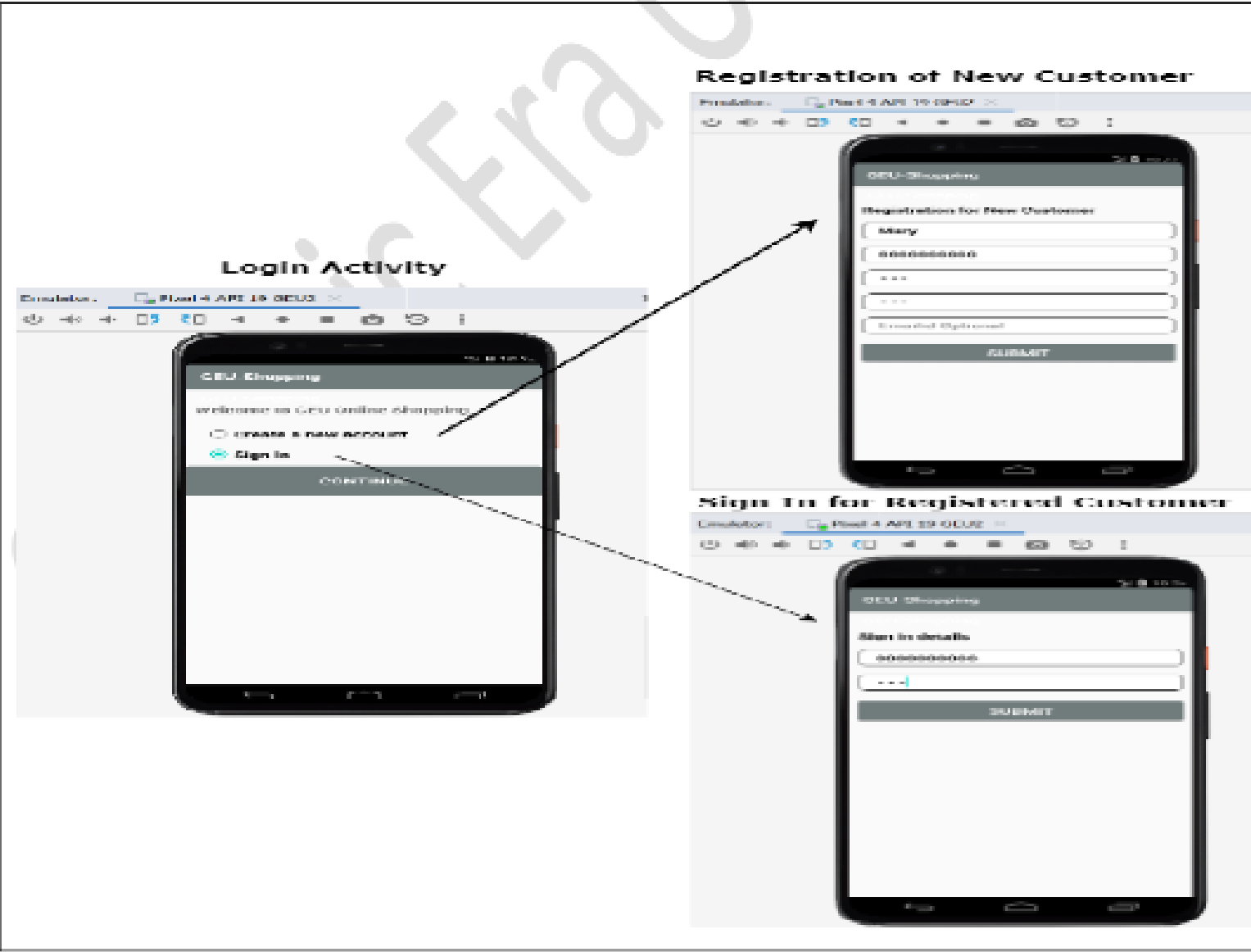


Figure 10.6: Integration testing of LoginActivity on Click of Radio button

The `setOnClickListener` and `onClick()` event handlers for 1st, as well as 2nd radio buttons, are tested and checked whether the appropriate activities get displayed.

```
// For new customer
RBnewlogin.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {    // Unit testing
        // creating Intent object
        Intent intent=new
Intent(LoginActivity.this,LoginNewActivity.class);
        startActivity(intent);    // starting another activity
    }
});
// For registered customer
RBregdusr.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {    // Unit testing
        // creating Intent object
        Intent intent=new
Intent(LoginActivity.this,LoginRegdActivity.class);
        startActivity(intent);    // starting another activity
    }
});
}
```

Figure 10.7:Event handler and EventListener in LoginActivity on Click of Radio buttons

System Testing: The full software application as a whole is checked to ensure that all the required features incorporated in the application are working/responsive and meet the specifications of the end users. System testing is done by the developers/testers before publishing the application in the market.

Acceptance Testing: This is the last level of software testing done by the end-user before accepting the end product/android application. Testing is usually done by two or more end users to check whether all the specifications given at the time of design have been met.

10.2.2 Types of Testing

There are various other types of testing done based on the environments and the type of android application developed before releasing to the users.

Functional Testing - The application is tested to check whether it meets the user's requirements.

User-Interface Testing - The application is tested on various devices for proper alignment of images, text visibility, scrolling of information wherever applicable, size of elements, etc.

Regression Testing - The application is tested when some modifications are done to the software to check whether the new functionality as well as the previous functionality is working properly.

Performance Testing - The application is tested to check for efficiency. For e.g. If the application under test is using a database, then it is verified whether data is retrieved quickly and efficiently.

Compatibility Testing – The application is run using various devices and API levels to check for the proper functioning of the application on all the devices used.

Interruption Testing – The application is tested for its functionality in an interrupted state. For e.g. The execution of an android app is tested for its proper and efficient working when the user is using the android app and is being interrupted by an incoming call or SMS, in this case, the app runs in the background and is later resumed to its state before the interruption.

Installation Testing – The application is checked for smooth installation without any errors during installation.

10.3 Android Testing Frameworks

Many Android Testing Frameworks are available in the market that can be used for testing android applications. A few are listed here

- **JUnit** – It generates XML reports and is an instrumental test runner
- **Appium** – It is an open-source framework that supports automated testing on real devices and emulators. It works for both Android OS and IOS. It supports many languages like JavaScript, Java, PHP, Ruby, Python, etc.
- **Detox** – It is JavaScript mobile testing framework used for testing an android application.
- **TestProject** - It is a free automation tool used for testing mobile applications. It is integrated with the browser and allows execution on any remote system which are registered.

- UI Automator - It is a GUI tool used for testing a user interface of an android application. It is used to inspect the hierarchy of the layouts and also the properties of the user-interface elements used.

10.4 Self-Assessment Questions

- Q1. Describe the levels of testing and android application in brief. [8 marks, L2]
- Q2. Define Unit testing. Explain unit testing with a suitable example.[6 marks, L2]
- Q3. Define Integration testing. Explain integration testing with a suitable example.
[6 marks, L2]
- Q4. Explain the types of testing used for testing an android application.[6 marks, L2]
- Q5. Explain the tools available for testing an android application.[5 marks, L2]
- Q6. Why is testing done? It is necessary to test an application before release. Justify your answer. [8 marks, L2]

10.5 Self-Assessment Activities

Note: Use an appropriate emulator for the execution of an android application.

- A1. Various android applications have been created as part of the activities in the previous chapters. Using any one of them discuss and demonstrate various types of testing that have been used in the application with suitable test cases.

[Unit testing, Integration testing, System testing, Functional testing, etc.]

10.6 Multiple-Choice Questions

1. The different levels of testing an android application are _____. [1 mark, L1]
 - a) Unit testing
 - b) Integration testing
 - c) System testing and Acceptance testing
 - d) All of the above
2. Testing of a small/single unit is known as _____. [1 mark, L1]
 - a) Unit testing
 - b) Integration testing
 - c) System testing
 - d) Functional Testing

3. Testing of more than one unit together is known as _____. [1 mark, L1]
- a) Unit testing
 - b) Integration testing
 - c) System testing
 - d) Functional Testing
4. Unit testing is done by _____. [1 mark, L1]
- a) Users
 - b) Developers
 - c) Designers
 - d) None of the above
5. System Testing is done by _____. [1 mark, L1]
- a) Developers/Testers
 - b) Users
 - c) Designers
 - d) None of the above
6. Acceptance testing is done by _____. [1 mark, L1]
- a) Developers
 - b) Designers
 - c) Testers
 - d) End Users
7. Functional testing is done to check whether _____. [1 mark, L1]
- a) it meets the user's requirements
 - b) it meets the developer's requirement
 - c) it meets the tester's requirement
 - d) all of the above
8. The application is run using various devices and API levels in _____. [1 mark, L1]
- a) Unit testing
 - b) Integration testing
 - c) Compatibility testing
 - d) None of the above

9. _____ is not an automated tool used for testing an android application.[1 mark, L1]
- a) JUnit
 - b) TestProject
 - c) Appium
 - d) Ruby
10. Regression testing is done when _____. [1 mark, L1]
- a) The software application is modified/updated
 - b) The software application is not modified/updated
 - c) The network issues exist
 - d) None of the above

0.7 Key Answers to multiple-choice questions

1. The different levels of testing an android application are Unit testing, Integration testing, System testing, and Acceptance testing. [d]
2. Testing of a small/single unit is known as Unit testing.[a]
3. Testing of more than one unit together is known as Integration testing.[b]
4. Unit testing is done by Developers.[b]
5. System Testing is done by Developers/Testers.[a]
6. Acceptance testing is done by end users.[d]
7. Functional testing is done to check whether **it meets the user's requirements**.[a]
8. The application is run using various devices and API levels in Compatibility testing.[c]
9. Ruby is not an automated tool used for testing an android application.[d]
10. Regression testing is done when the software application is modified/updated.[a]

0.8 Summary

Any software development is incomplete without testing. In a software application development process, testing is one of its phases. Any software application developed is tested before it is handed over to the end user. Testing is done to find out if any error(s) exists. The errors identified are fixed and checked to deliver an application of quality to the users. To assure and maintain the quality of the software, the application is tested at various levels. The different levels of testing are - Unit Testing, Integration Testing, System Testing, and Acceptance Testing.

Unit Testing - Testing of small unit/program code is called as unit testing.

Integration Testing: After testing the small modules, these modules are integrated and then tested to check whether the code gets executed properly.

System Testing: The full software application as a whole is checked to ensure that all the required features incorporated in the application are working/responsive and meet the specifications of the end users.

Acceptance Testing: This is the last level of software testing done by the end user before accepting the end product/android application.

There are various other types of testing done based on the environments and the type of android application developed before releasing to the users. They are - Functional Testing, User-Interface Testing, Regression Testing, Performance Testing, Compatibility Testing, Interruption Testing, Installation Testing, etc.

Many Android Testing Frameworks are available in the market that can be used for testing android applications. Some of them are Junit, Appium, Detox, TestProject, and UI Automator, etc.

10.9 Keywords

- Unit testing
- Integration testing
- System testing
- Acceptance testing
- Regression testing

10.10 Recommended Resources for Further Reading

Text Book(s)

1. Pradeep Kothari, "Android Application Development" Black-Book, Dreamtech Press, 2015
2. Wei-Meng Lee, "Beginning Android Application Development", Wiley 2011.
3. Dawn Griffiths and David Griffiths, "Head First Android Development", O'Reilly, 2017.

References

- <https://www.javatpoint.com/android-tutorial>
- <https://www.tutorialspoint.com/android/index.htm>
- <https://developer.android.com/guide/components/fundamentals>